Document-Aware KV-Cache Management for Repeated Long-Document Question Answering

September 2026

[Barbalata Radu Ionut - 2002688]

Abstract

Answering several questions about one document makes a language model process much of the same input repeatedly. I study whether caching each document's KV state as one entry can reduce this work and simplify cache management. I compare document, fixed-block and radix caches with Qwen2.5-1.5B on QuALITY, where each question supplies its article. Over 2,086 requests, document/LRU reduces mean time to first token (TTFT) by 21.9% with random ordering and 63.5% with Zipf traffic. CPU INT8 increases capacity but changes 12 answers on the random trace. In smaller experiments, a page arena does not improve latency in the Transformers runner, while a Triton kernel speeds up INT8 restoration by $1.209\times$ after warm-up.

1. Introduction and scope

An assistant answering questions about a long article receives nearly the same prompt each time: the article stays, while the question changes, and Transformers retain attention keys and values during generation (Vaswani et al., 2017). Reusing this state across requests with an exact common prefix avoids processing the article again, but keeping every article's KV state may not fit in memory, so some entries must be evicted.

I initially proposed studying this in RAG (Lewis et al., 2020): requests would use two or three documents, with the same combinations recurring across questions. A company legal assistant might repeatedly need the same statutory provisions and internal policy. I did not find a ready-made public serving trace with this pattern, and constructing one would require choosing document combinations, their order and repetition frequency. I therefore use QuALITY, where each question supplies its `article_id` (Pang et al., 2022), to study reuse of one known document first. Retrieval and multi-document prompts are outside this experiment.

Email: [Barbalata Radu Ionut - 2002688] <[barbalata.2002688@studenti.uniroma1.it]>.

Deep Learning and Applied AI 2026, Sapienza University of Rome, 2nd semester a.y. 2025/2026.

2. Method

Every prompt uses the same order:

$$\underbrace{\text{system prefix}}_{L0} \parallel \underbrace{\text{article}}_{\text{reusable}} \parallel \underbrace{\text{question + A/B/C/D}}_{\text{request-specific}}.$$

L0 is never evicted or counted as article reuse; questions and options are never cached. For T tokens, L layers, H_{kv} KV heads, head width D and b bytes per value, the KV size is $B_{KV}(T) = 2LH_{kv}DTb$. Qwen2.5-1.5B has 28 layers, two KV heads and width 128: each FP16 token therefore costs 28 KiB (Yang et al., 2024).

All three caches use the same Transformers runner. **Document** stores and evicts whole articles, **fixed-block** uses 256-token blocks inspired by vLLM (Kwon et al., 2023), and **radix** uses longest-prefix lookup inspired by SGLang (Zheng et al., 2024). These are local implementations, not benchmarks of either engine. The reported runs use LRU, with GDSF added for Zipf. KV tensors use GPU FP16 or symmetric CPU INT8 with per-layer, per-KV-head scales. Restoring CPU INT8 entries requires transfer and dequantization, so TTFT includes both costs.

To compare how much useful work each cache avoids, I count article tokens:

$$h_{article} = \frac{\sum_i \text{restored article tokens}_i}{\sum_i \text{requested article tokens}_i}. \tag{1}$$

The no-cache baseline processes the same system, article and suffix stages, but discards article KV after every request. Using this segmented baseline matters: otherwise, differences between one full forward pass and several smaller passes could be mistaken for a benefit of caching.

Table 1. Selected full-dev inference paths: 2,086 requests per path, 4 GiB for cached paths. TTFT mean and p90 are in seconds; speedup compares means with the aligned segmented control. h excludes L0. Storage is accelerator FP16 except CPU INT8/Triton.

Traffic	Cache	$h_{article}$	Mean	p90	Speedup
Random	Segmented	0.00%	1.843	2.623	1.000×
Random	Document/LRU	22.94%	1.439	2.562	1.281×
Random	Fixed-256/LRU	23.15%	1.474	2.605	1.251×
Random	Radix/LRU	22.77%	1.528	2.736	1.206×
Random	Document/INT8	44.67%	1.168	2.752	1.578×
Zipf	Segmented	0.00%	2.208	2.670	1.000×
Zipf	Document/LRU	65.38%	0.806	2.543	2.741×
Zipf	Fixed-256/LRU	65.27%	0.863	2.566	2.558×
Zipf	Radix/LRU	65.23%	0.762	2.457	2.898×
Zipf	Document/GDSF	71.46%	0.633	2.310	3.487×
Zipf	Document/INT8	81.66%	0.455	2.207	4.851×

3. Experimental protocol

The data is HTML-stripped QuALITY v1.0.1. Merging the two writer records per article preserves all 381 articles and 6,737 questions. Accuracy is measured on dev; the test split is used only for cache traces because its labels are withheld. With seed 42, random traffic shuffles the questions, while Zipf favors popular articles and cycles through their questions. Answers are selected by scoring A/B/C/D, with sequence-likelihood fallback when a label is not a single token.

I ran twelve Qwen2.5-1.5B-Instruct configurations on a Tesla T4. Each serves 2,086 aligned requests, with 4 GiB for cached runs. Random visits every dev question once; Zipf contains 911 distinct Q&A, so I also remove duplicates when evaluating accuracy. A 108-run no-inference matrix selected the cache strategies and policies. Code and experiment details are at `https://github.com/i0nut02/rag-kvcache`.

4. Results

Cache organization. Table 1 shows that caching reduces average latency in both workloads, with larger savings under Zipf because popular articles are requested repeatedly. The choice of cache structure makes a smaller difference: document and radix differ by less than 0.5% in mean TTFT across the repeated runs, whereas fixed-256 is slower in every repetition. Document's clearer advantage is the amount of bookkeeping it avoids by managing each article as one entry, using about 22 KB of metadata compared with 5.72 MB for fixed-256 on full random dev.

These comparisons also show why a partial hit is useful only in proportion to how much text it restores. On Zipf, the median partial hit covers 6,400 article tokens with fixed blocks but only one token with radix, so counting both as a hit would hide a large difference in avoided prefill work. The token-weighted metric captures this distinction, while p90 shows the remaining cost of misses, which still require processing most of the article. Eviction policy matters when some articles are more popular than others: on Zipf, GDSF raises article-token reuse from 65.38% to 71.46% and lowers mean TTFT by 21.4% compared with document/LRU.

INT8 and answer accuracy. INT8 trades numerical precision for enough space to keep more articles, with 4 GiB nearly matching the capacity of 8 GiB of FP16 in simulation. On random traffic this improves mean latency, though p90 becomes worse, and unchanged overall accuracy hides 12 answer changes: six become correct and six become wrong. Zipf needs particular care because one helpful change is counted 32 times as the same question recurs; counting each unique Q&A once instead gives 496 correct answers for INT8 versus 499 for the baseline, out of 911. Across these unique items, 14 labels change: seven correct-to-wrong, four wrong-to-correct and three wrong-to-different-wrong. The useful gain is therefore cache capacity, not better answer quality, even when aggregate accuracy appears unchanged or higher.

Arena and Triton experiments (100 requests). The arena and Triton experiments illustrate how an optimization of cache storage does not necessarily improve the whole runner. In random-dev tests with the same model, T4 and seed, both arena page sizes increase latency because Transformers still requires contiguous KV tensors to be rebuilt before use. Triton has a more direct benefit in INT8 restoration (Tillet et al., 2019): at 31 matched cache-hit positions, complete restore falls from 37.45 to 30.98 ms with identical scores to PyTorch INT8. That saving applies after warmup and must first recover the 2.671 s JIT startup cost. At the observed 31% hit rate, the estimated break-even point is roughly 1,300 requests, using a simple extrapolation detailed in the repository rather than a measured crossover.

Full run-level tables, three-repetition ranges, accuracy breakdowns and the arena/Triton microbenchmarks are available in `docs/results.md` and `docs/generated/` at the repository URL above.

5. Discussion and conclusion

For this workload, document/LRU combines low management cost with latency close to radix, while fixed-block is consistently slower after tuning. The evidence covers QuALITY, two Qwen sizes and one T4; full configurations ran once and shorter comparisons ran three times on one seed, so they show timing variation rather than a gen-

eral ranking. RAG would also require versioned retrieved prompts and separate retrieval timing.

6. Statement on the use of AI

The main ideas and initial project structure were my own. I used generative AI extensively to develop the implementation: the Triton kernel, experiment scripts and much of the supporting test and analysis code were mainly AI-written. AI also helped me refine the design, read the literature and refactor code.

I used AI to prepare tables and plots from saved results, discuss their interpretation, check cache correctness and revise this report. This included spotting unfair timing comparisons and planning further tests. I reviewed the generated code and checked the tests and saved outputs during development.

References

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles*, 2023. doi: 10.1145/3600006.3613165. URL https://arxiv.org/abs/2309.06180.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuettler, H., Lewis, M., Yih, W.-t., Rocktaeschel, T., Riedel, S., and Kiela, D. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, 2020. URL https://arxiv.org/abs/2005.11401.

Pang, R. Y., Parrish, A., Joshi, N., Nangia, N., Phang, J., Chen, A., Padmakumar, V., Ma, J., Thompson, J., He, H., and Bowman, S. R. QuALITY: Question answering with long input texts, yes! In *Proceedings of NAACL-HLT*, 2022. URL https://arxiv.org/abs/2112.08608.

Tillet, P., Kung, H. T., and Cox, D. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pp. 10–19, 2019. doi: 10.1145/3315508.3329973.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://arxiv.org/abs/1706.03762.

Yang, A. et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024. URL https://arxiv.org/abs/2412.15115.

Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., and Sheng, Y. SGLang: Efficient execution of structured language model programs. *arXiv preprint arXiv:2312.07104*, 2024. URL https://arxiv.org/abs/2312.07104.